

DATABASE DESIGN

FINAL PROJECT REPORT

CAR RENTAL SYSTEM

TEAM MEMBERS:

A.Sai Akhil - AP24110010558

V . Kishan Kireeti – AP24110010575



INDEX

1. Introduction	3
2. Entities	4
3. Relations	5
4. Assumptions	6
5. ER Diagram	7
6. Functional Dependencies	8-9
7. Final Relational Schema	10
8. SQL Statements	11-14
9. Views	15-16
10. Conclusion	17

1. INTRODUCTION

We have chosen to design and implement a Car Rental System database. In our system, a customer can rent a car by selecting the desired vehicle from the available fleet at a branch. The system manages bookings on both an hourly and daily basis, tracks vehicle details such as fuel type, transmission and engine specifications, handles payment transactions, records pre- and post-rental inspections, and logs all maintenance activities for each car.

Our system is designed for a rental business operating from multiple branches. Each branch maintains its own fleet of cars and staff. The system enforces data integrity through primary keys, foreign keys, and check constraints to ensure only valid data is stored.

1.1 REQUIREMENTS

- a. Each booking is associated with only one customer at a time.
- b. Each booking is associated with only one car at a time.
- c. A car must belong to a branch (branch is mandatory for a car).
- d. Not all bookings may have a fine amount — fine is only applied for late returns.
- e. Booking may or may not have an inspection record depending on whether inspection was conducted.
- f. A car may or may not have maintenance records — new cars may have none yet.
- g. Payment may or may not exist for a booking because cancelled bookings may not be paid.
- h. A customer can have multiple bookings over time, but each booking belongs to exactly one customer.
- i. A car can appear in multiple bookings over time, but each booking is for exactly one car.
- j. Inspection may be of type pre-rental or post-rental linked to the same booking.
- k. A customer with blacklist status should not be allowed to make new bookings.
- l. Billing details such as advance payment and caution deposit are stored per booking, not per customer.

2. ENTITIES

- a) **Branch:** Branch represents a physical office location of the car rental business. Each branch has a branch name and location. All cars and employees are linked to a branch.
- b) **Customer:** Customer is the person who rents a car. The entity stores the customer's name, phone number, email, address, Aadhaar number (unique), driving licence number (unique), and a blacklist status flag to prevent misuse.
- c) **Car:** Car represents a vehicle in the fleet. It holds detailed technical attributes — brand, model, manufacturing year, car type, fuel type (petrol/diesel/electric), transmission type, engine capacity, horsepower, torque, airbag count, mileage, battery range (for EVs), purpose type (city/highway), pricing (per day and per hour), security deposit, current status, and the branch it belongs to.
- d) **Employee:** Employee represents staff at a branch. It records the employee's name, role, and phone number, and links them to their assigned branch.
- e) **Booking:** Booking captures each rental transaction. It records the customer, the car, booking date, rental start and end datetimes, total hours and days, advance payment, caution deposit, computed total amount, any fine, the final payable amount, and the booking status.
- f) **Payment:** Payment records each financial transaction linked to a booking, including the payment method, payment date, amount paid, and its status (Paid, Pending, or Failed).
- g) **Maintenance:** Maintenance logs all servicing events for a vehicle. It captures the service date, service type, description, service cost, the service center name, and the next scheduled service date.
- h) **Inspection:** Inspection documents the condition of a car before it is handed over and after it is returned. It records the inspection type, fuel level, odometer reading, any damage notes, and optional image proof. Each inspection is linked to both a booking and a car.

3. RELATIONS

- i) **Branch to Car:** Each car is assigned to one branch. A branch can hold many cars. This is a one-to-many relationship. The relation name is 'Assigned To'.
- j) **Branch to Employee:** Each employee works at one branch. A branch can have many employees. This is a one-to-many relationship. The relation name is 'Works At'.
- k) **Customer to Booking:** A customer can make multiple bookings over time, but each booking is associated with exactly one customer. This is a one-to-many relationship. The relation name is 'Makes'.
- l) **Car to Booking:** A car can appear in many bookings over its lifetime (once returned, it becomes available again), but each booking is for exactly one car. This is a one-to-many relationship. The relation name is 'Reserved In'.
- m) **Booking to Payment:** A booking can have one or more associated payment entries (advance, balance, fine, etc.). Each payment belongs to one booking. This is a one-to-many relationship. The relation name is 'Settled Through'.
- n) **Car to Maintenance:** A car can have many maintenance records over its operational life. Each maintenance record belongs to one car. This is a one-to-many relationship. The relation name is 'Serviced In'.
- o) **Booking to Inspection:** A booking can have one or two inspection records (pre-delivery and post-return). Each inspection belongs to one booking and references one car. The relation name is 'Inspected Under'.

4. ASSUMPTIONS

- p. Each payment is linked to exactly one booking, but a booking can have multiple payments (e.g., advance and final payment).
- q. A car can only have one status at a time — either Available, Booked, or Maintenance — as enforced by the status check constraint.
- r. Every car must be registered under a branch; a car cannot exist in the system without a branch assignment.
- s. Security deposit is stored per car, not per booking, meaning it is fixed based on the car regardless of rental duration.
- t. An employee must belong to exactly one branch; an employee cannot exist without a branch assignment in the system.
- u. A customer must have a unique Aadhaar number and a unique driving license number — no two customers can share these identifiers.
- v. Total hours and total days are both stored separately in the booking table to support both hourly and daily rental pricing models.
- w. Maintenance records are independent of bookings — a car can be sent for maintenance without any active or past booking.
- x. Inspection records require both a booking ID and a car ID, meaning an inspection can only be conducted in the context of a booking.
- y. The system supports both petrol, diesel, and electric vehicles since battery range and mileage are stored as separate fields in the car table.

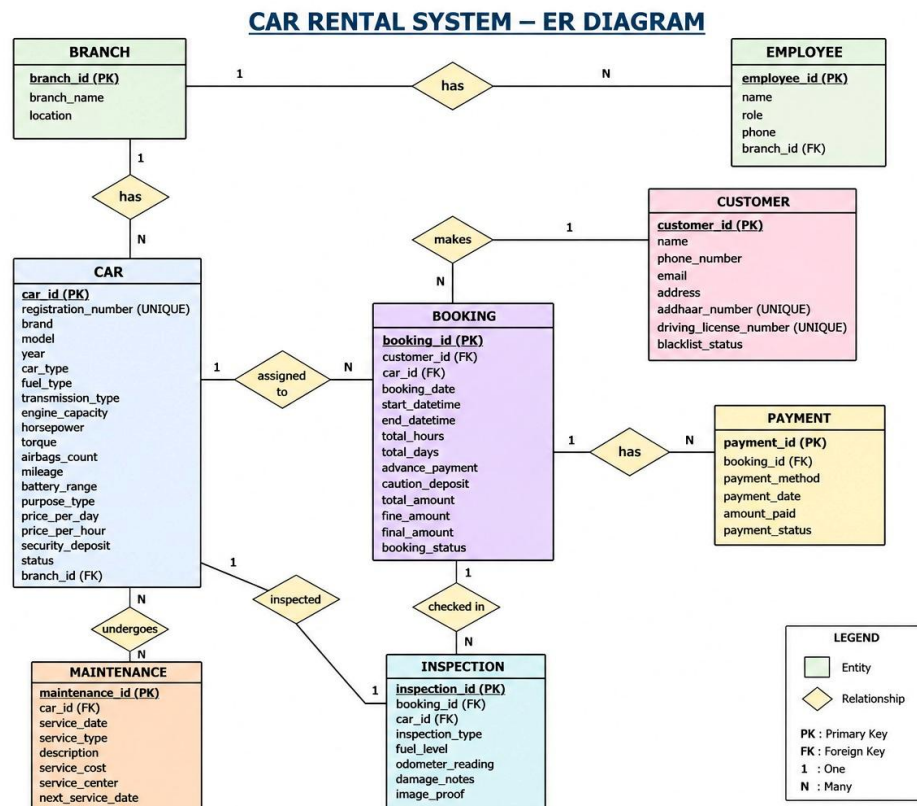
5. ER DIAGRAM

The ER diagram below represents all the entities, their attributes, and the relationships between them in the Car Rental System. Entities are shown as rectangles, attributes as ovals, and relationships as diamonds. Primary keys are underlined. Foreign key references are shown as directional arrows.

Entities in the diagram: Branch, Customer, Car, Employee, Booking, Payment, Maintenance, Inspection.

Key relationships with cardinality:

- Branch (1) — Assigned To — (N) Car
- Branch (1) — Works At — (N) Employee
- Customer (1) — Makes — (N) Booking
- Car (1) — Reserved In — (N) Booking
- Booking (1) — Settled Through — (N) Payment
- Car (1) — Serviced In — (N) Maintenance
- Booking (1) — Inspected Under — (N) Inspection



6. FUNCTIONAL DEPENDENCIES

p) Branch Relation:

branch_id -> branch_name, location

q) Customer Relation:

customer_id -> name, phone_number, email, address,
aadhaar_number, driving_license_number,
blacklist_status
aadhaar_number -> customer_id, name, phone_number, email
driving_license_number -> customer_id, name

r) Car Relation:

car_id -> registration_number, brand, model, year,
car_type, fuel_type, transmission_type,
engine_capacity, horsepower, torque,
airbags_count, mileage, battery_range,
purpose_type, price_per_day, price_per_hour,
security_deposit, status, branch_id
registration_number -> car_id (alternate key)

s) Employee Relation:

employee_id -> name, role, phone, branch_id

t) Booking Relation:

booking_id -> customer_id, car_id, booking_date, start_datetime,
end_datetime, total_hours, total_days,
advance_payment, caution_deposit, total_amount,
fine_amount, final_amount, booking_status

u) Payment Relation:

payment_id -> booking_id, payment_method, payment_date,
amount_paid, payment_status

v) Maintenance Relation:

maintenance_id -> car_id, service_date, service_type,
description, service_cost,
service_center, next_service_date

w) Inspection Relation:

inspection_id -> booking_id, car_id, inspection_type,
fuel_level, odometer_reading,
damage_notes, image_proof

6.1 Functional Dependencies That Violate Normalization Rules

The following transitive dependencies exist in the relational schema:

- **Car Relation:**
car_id -> brand
brand -> model (if a specific model uniquely implies a brand)
- **Customer Relation:**
customer_id -> aadhaar_number
customer_id -> driving_license_number
(both are candidate keys – BCNF consideration)
- **Car Relation (branch address):**
car_id -> branch_id
branch_id -> location
(resolved by keeping Branch as a separate table)

The schema is maintained in 3NF. The Branch table is separated to eliminate the transitive dependency car_id -> branch_id -> location.

7. FINAL RELATIONAL SCHEMA

The final relational schema is represented in 3NF. The Branch table eliminates transitive dependencies between car details and branch location. Primary keys are underlined; foreign keys reference the primary keys of their respective parent tables.

Table	Attributes (PK underlined, FK noted)
Branch	<u>branch_id</u> (PK), branch_name, location
Customer	<u>customer_id</u> (PK), name, phone_number, email, address, aadhaar_number (UNIQUE), driving_license_number (UNIQUE), blacklist_status
Car	<u>car_id</u> (PK), registration_number (UNIQUE), brand, model, year, car_type, fuel_type, transmission_type, engine_capacity, horsepower, torque, airbags_count, mileage, battery_range, purpose_type, price_per_day, price_per_hour, security_deposit, status, branch_id (FK -> Branch)
Employee	<u>employee_id</u> (PK), name, role, phone, branch_id (FK -> Branch)
Booking	<u>booking_id</u> (PK), customer_id (FK -> Customer), car_id (FK -> Car), booking_date, start_datetime, end_datetime, total_hours, total_days, advance_payment, caution_deposit, total_amount, fine_amount, final_amount, booking_status
Payment	<u>payment_id</u> (PK), booking_id (FK -> Booking), payment_method, payment_date, amount_paid, payment_status
Maintenance	<u>maintenance_id</u> (PK), car_id (FK -> Car), service_date, service_type, description, service_cost, service_center, next_service_date
Inspection	<u>inspection_id</u> (PK), booking_id (FK -> Booking), car_id (FK -> Car), inspection_type, fuel_level, odometer_reading, damage_notes, image_proof

8. SQL STATEMENTS

8.1 Create Database

```
CREATE DATABASE car_rental_system;  
USE car_rental_system;
```

8.2 Create Table Statements

x) Branch

```
CREATE TABLE Branch (  
    branch_id    INT PRIMARY KEY AUTO_INCREMENT,  
    branch_name  VARCHAR(100),  
    location     VARCHAR(100)  
);
```

y) Customer

```
CREATE TABLE Customer (  
    customer_id    INT PRIMARY KEY AUTO_INCREMENT,  
    name           VARCHAR(100),  
    phone_number   VARCHAR(15),  
    email          VARCHAR(100),  
    address        TEXT,  
    aadhaar_number VARCHAR(20) UNIQUE,  
    driving_license_number VARCHAR(50) UNIQUE,  
    blacklist_status    BOOLEAN DEFAULT FALSE,  
    CHECK (phone_number IS NULL OR  
           LENGTH(phone_number) BETWEEN 10 AND 15),  
    CHECK (email LIKE '%@%.%')  
);
```

z) Car

```
CREATE TABLE Car (  
    car_id            INT PRIMARY KEY AUTO_INCREMENT,  
    registration_number VARCHAR(20) UNIQUE,  
    brand             VARCHAR(50),  
    model             VARCHAR(50),  
    year              INT,  
    car_type          VARCHAR(20),  
    fuel_type         VARCHAR(20),  
    transmission_type VARCHAR(20),  
    engine_capacity   INT,  
    horsepower        INT,  
    torque            INT,  
    airbags_count     INT,  
    mileage            DECIMAL(5,2),  
    battery_range      DECIMAL(5,2),  
    purpose_type       VARCHAR(20),  
    price_per_day      DECIMAL(10,2),
```

```
price_per_hour      DECIMAL(10,2),
security_deposit    DECIMAL(10,2),
status              VARCHAR(20),
branch_id           INT,
FOREIGN KEY (branch_id) REFERENCES Branch(branch_id),
CHECK (year >= 2000),
CHECK (price_per_day > 0 AND price_per_hour > 0),
CHECK (status IN ('Available','Booked','Maintenance'))
);
```

aa) Employee

```
CREATE TABLE Employee (
    employee_id INT PRIMARY KEY AUTO_INCREMENT,
    name        VARCHAR(100),
    role        VARCHAR(50),
    phone       VARCHAR(15),
    branch_id   INT,
    FOREIGN KEY (branch_id) REFERENCES Branch(branch_id)
);
```

bb) Booking

```
CREATE TABLE Booking (
    booking_id      INT PRIMARY KEY AUTO_INCREMENT,
    customer_id     INT,
    car_id          INT,
    booking_date    DATE,
    start_datetime  DATETIME,
    end_datetime    DATETIME,
    total_hours     INT,
    total_days      INT,
    advance_payment DECIMAL(10,2),
    caution_deposit DECIMAL(10,2),
    total_amount    DECIMAL(10,2),
    fine_amount     DECIMAL(10,2),
    final_amount    DECIMAL(10,2),
    booking_status  VARCHAR(20),
    FOREIGN KEY (customer_id)
        REFERENCES Customer(customer_id),
    FOREIGN KEY (car_id) REFERENCES Car(car_id),
    CHECK (end_datetime > start_datetime),
    CHECK (total_amount >= 0 AND final_amount >= 0)
);
```

cc) Payment

```
CREATE TABLE Payment (
    payment_id      INT PRIMARY KEY AUTO_INCREMENT,
    booking_id      INT,
    payment_method  VARCHAR(20),
    payment_date    DATE,
```

```
amount_paid    DECIMAL(10,2),
payment_status VARCHAR(20),
FOREIGN KEY (booking_id) REFERENCES Booking(booking_id),
CHECK (amount_paid > 0),
CHECK (payment_status IN ('Paid','Pending','Failed'))
);
```

dd) Maintenance

```
CREATE TABLE Maintenance (
    maintenance_id INT PRIMARY KEY AUTO_INCREMENT,
    car_id          INT,
    service_date    DATE,
    service_type    VARCHAR(50),
    description     TEXT,
    service_cost    DECIMAL(10,2),
    service_center  VARCHAR(100),
    next_service_date DATE,
    FOREIGN KEY (car_id) REFERENCES Car(car_id),
    CHECK (service_cost >= 0)
);
```

ee) Inspection

```
CREATE TABLE Inspection (
    inspection_id INT PRIMARY KEY AUTO_INCREMENT,
    booking_id    INT,
    car_id        INT,
    inspection_type VARCHAR(20),
    fuel_level    DECIMAL(5,2),
    odometer_reading INT,
    damage_notes  TEXT,
    image_proof   VARCHAR(255),
    FOREIGN KEY (booking_id) REFERENCES Booking(booking_id),
    FOREIGN KEY (car_id) REFERENCES Car(car_id),
    CHECK (fuel_level >= 0 AND fuel_level <= 100)
);
```

8.3 Insert SQL Statements

ff) Branch

```
INSERT INTO Branch (branch_name, location)
VALUES ('Vijayawada Branch', 'Vijayawada');
```

gg) Customer

```
INSERT INTO Customer (name, phone_number, email, address,
                      aadhaar_number, driving_license_number)
VALUES ('Sai Akhil', '9876543210', 'akhil@gmail.com',
        'AP', '123456789012', 'DL12345');
```

hh) Car

```
INSERT INTO Car (registration_number, brand, model, year,
    car_type, fuel_type, transmission_type, engine_capacity,
    horsepower, torque, airbags_count, mileage, purpose_type,
    price_per_day, price_per_hour, security_deposit,
    status, branch_id)
VALUES ('AP16AB1234', 'Toyota', 'Innova', 2022, 'SUV',
    'Diesel', 'Manual', 2400, 150, 350, 6, 15.0, 'Highway',
    2500, 200, 10000, 'Available', 1);
```

ii) Booking

```
INSERT INTO Booking (customer_id, car_id, booking_date,
    start_datetime, end_datetime, total_days,
    advance_payment, caution_deposit,
    total_amount, final_amount, booking_status)
VALUES (1, 1, CURDATE(),
    '2026-04-20 10:00:00', '2026-04-23 10:00:00',
    3, 2000, 10000, 7500, 5500, 'Confirmed');
```

jj) Payment

```
INSERT INTO Payment (booking_id, payment_method,
    payment_date, amount_paid, payment_status)
VALUES (1, 'UPI', CURDATE(), 2000, 'Paid');
```

9. VIEWS

Based on the data stored across the tables, we identified the following business reports that would be useful in day-to-day operations of the car rental system. Each view represents one such report and is implemented as an SQL view for reusability.

kk) Available Cars View

This view lists all cars that are currently available for booking, showing the car ID, brand, model, and daily price.

```
CREATE VIEW vw_available_cars AS
SELECT car_id, brand, model, price_per_day
FROM Car
WHERE status = 'Available';
```

ll) Booking Details View

This view joins the Booking, Customer, and Car tables to produce a complete booking summary showing the customer name, car details, rental period, final amount, and booking status.

```
CREATE VIEW vw_booking_details AS
SELECT
    b.booking_id,
    c.name          AS customer_name,
    car.brand,
    car.model,
    b.start_datetime,
    b.end_datetime,
    b.final_amount,
    b.booking_status
FROM Booking b
JOIN Customer c    ON b.customer_id = c.customer_id
JOIN Car car       ON b.car_id       = car.car_id;
```

mm) Payment Summary View

This view joins the Payment and Booking tables to show each payment along with its corresponding booking, method used, amount, and payment status.

```
CREATE VIEW vw_payment_summary AS
SELECT
    p.payment_id,
    b.booking_id,
    p.amount_paid,
    p.payment_method,
    p.payment_status
FROM Payment p
JOIN Booking b ON p.booking_id = b.booking_id;
```

nn) Maintenance History View

This view displays the maintenance history for all cars, joining the Maintenance and Car tables to show the registration number, service date, service type, and cost.

```
CREATE VIEW vw_maintenance_history AS
SELECT
    c.registration_number,
    m.service_date,
    m.service_type,
    m.service_cost
FROM Maintenance m
JOIN Car c ON m.car_id = c.car_id;
```

oo) Inspection Report View

This view produces an inspection report by joining the Inspection and Booking tables, showing the inspection ID, booking ID, inspection type, fuel level, and odometer reading.

```
CREATE VIEW vw_inspection_report AS
SELECT
    i.inspection_id,
    b.booking_id,
    i.inspection_type,
    i.fuel_level,
    i.odometer_reading
FROM Inspection i
JOIN Booking b ON i.booking_id = b.booking_id;
```

10. CONCLUSION

During the course of this project, we gained hands-on experience in the complete lifecycle of database design and implementation. We started from system requirement analysis, identified the entities, their attributes and the relationships between them, and translated these into a proper ER diagram.

We then converted the ER diagram into relational tables and applied normalization rules to arrive at a 3NF schema. The functional dependencies were carefully analysed and transitive dependencies were eliminated by decomposing tables appropriately — for example, keeping Branch as a separate entity to avoid redundant storage of location data inside the Car table.

The implementation phase involved writing DDL statements to create all eight tables in MySQL with appropriate primary keys, foreign keys, unique constraints, and check constraints. We inserted valid sample data and designed five views to generate business reports such as available car listings, booking summaries, payment records, maintenance history, and inspection reports.

Overall, this project gave us a thorough understanding of how to design a clean, normalized database system for a real-world application and how to implement and query it efficiently using SQL.